Laboratory Work 7

SQL Views and Roles

Objective: Create regular and materialized views, modify data through views, and manage PostgreSQL access with roles.

1 Database setup

Use the Lab 6 tables: employees(emp_id, emp_name, dept_id, salary), departments(dept_id, dept_name, location), projects(project_id, project_name, dept_id, budget). Run the exercises in order as postgres in a lab database. Run expected-error statements separately with autocommit enabled.

```
SET search_path TO public;
```

2 Basic views

2 1 Employee details

```
CREATE VIEW employee_details AS
SELECT e.emp_name, e.salary, d.dept_name, d.location
FROM employees e JOIN departments d ON e.dept_id = d.dept_id;
SELECT * FROM employee_details;
SELECT COUNT(*) FROM employee_details;
```

Answer: The count equals the number of employees with a matching department. Tom Brown is excluded if his dept_id is NULL or has no match. The exact count depends on Lab 6 data.

2 2 Department statistics

```
CREATE VIEW dept_statistics AS
SELECT d.dept_name, COUNT(e.emp_id) AS employee_count,
       AVG(e.salary) AS avg_salary, MAX(e.salary) AS max_salary,
       MIN(e.salary) AS min_salary
FROM departments d LEFT JOIN employees e USING (dept_id)
GROUP BY d.dept_id, d.dept_name;
SELECT * FROM dept_statistics ORDER BY employee_count DESC;
```

2 3 Project overview

```
CREATE VIEW project_overview AS
SELECT p.project_name, p.budget, d.dept_name, d.location,
       COUNT(e.emp_id) AS team_size
FROM projects p LEFT JOIN departments d USING (dept_id)
LEFT JOIN employees e ON e.dept_id = d.dept_id
GROUP BY p.project_id, p.project_name, p.budget,
         d.dept_name, d.location;
```

2 4 High earners

```
CREATE VIEW high_earners AS
SELECT e.emp_name, e.salary, d.dept_name
FROM employees e LEFT JOIN departments d USING (dept_id)
WHERE e.salary > 55000;
SELECT * FROM high_earners;
```

Answer: All employees earning above 55,000 appear, including those without a department; their department name is NULL.

3 Modifying views

3 1 Replace employee details

```
CREATE OR REPLACE VIEW employee_details AS
SELECT e.emp_name, e.salary, d.dept_name, d.location,
       CASE WHEN e.salary > 60000 THEN 'High'
            WHEN e.salary > 50000 THEN 'Medium'
            ELSE 'Standard' END AS salary_grade
FROM employees e JOIN departments d USING (dept_id);
```

3 2 Rename a view

```
ALTER VIEW high_earners RENAME TO top_performers;
SELECT * FROM top_performers;
```

3 3 Create and drop a temporary view

```
CREATE TEMP VIEW temp_view AS
SELECT * FROM employees WHERE salary < 50000;
DROP VIEW temp_view;
```

4 Updatable views

4 1 Create an updatable view

```
CREATE VIEW employee_salaries AS
SELECT emp_id, emp_name, dept_id, salary FROM employees;
```

4 2 Update through the view

```
UPDATE employee_salaries SET salary = 52000
WHERE emp_name = 'John Smith';
SELECT * FROM employees WHERE emp_name = 'John Smith';
```

Answer: Yes. Updating this simple view updates the underlying employees table.

4 3 Insert through the view

```
INSERT INTO employee_salaries (emp_id, emp_name, dept_id, salary)
VALUES (6, 'Alice Johnson', 102, 58000);
SELECT * FROM employees WHERE emp_id = 6;
```

Answer: The insert succeeds if ID 6 is unused, department 102 exists, and other table constraints are satisfied.

4 4 Local check option

```
CREATE VIEW it_employees AS
SELECT emp_id, emp_name, dept_id, salary
FROM employees WHERE dept_id = 101
WITH LOCAL CHECK OPTION;
-- Run this expected-error test separately:
INSERT INTO it_employees (emp_id, emp_name, dept_id, salary)
VALUES (7, 'Bob Wilson', 103, 60000);
```

Expected error: new row violates check option for view "it_employees". Department 103 does not satisfy dept_id = 101.

5 Materialized views

5 1 Department summary

```
CREATE MATERIALIZED VIEW dept_summary_mv AS
SELECT d.dept_id, d.dept_name,
       COALESCE(e.n, 0) AS total_employees,
       COALESCE(e.pay, 0) AS total_salaries,
       COALESCE(p.n, 0) AS total_projects,
       COALESCE(p.budget, 0) AS total_project_budget
FROM departments d
LEFT JOIN (
  SELECT dept_id, COUNT(*) AS n, SUM(salary) AS pay
  FROM employees GROUP BY dept_id
) e USING (dept_id)
LEFT JOIN (
  SELECT dept_id, COUNT(*) AS n, SUM(budget) AS budget
  FROM projects GROUP BY dept_id
) p USING (dept_id)
WITH DATA;
SELECT * FROM dept_summary_mv ORDER BY total_employees DESC;
```

Employees and projects are aggregated separately to prevent duplicated counts and totals.

5 2 Refresh the cached data

```
INSERT INTO employees (emp_id, emp_name, dept_id, salary)
VALUES (8, 'Charlie Brown', 101, 54000);
SELECT * FROM dept_summary_mv WHERE dept_id = 101;
REFRESH MATERIALIZED VIEW dept_summary_mv;
SELECT * FROM dept_summary_mv WHERE dept_id = 101;
```

Answer: Before refresh, the old totals remain. After refresh, IT gains one employee and total salaries increase by 54,000, assuming no other changes.

5 3 Concurrent refresh

```
CREATE UNIQUE INDEX dept_summary_mv_dept_id_idx
ON dept_summary_mv (dept_id);
REFRESH MATERIALIZED VIEW CONCURRENTLY dept_summary_mv;
```

Answer: Other sessions can keep reading the view during refresh. It must already contain data and have a qualifying unique index.

5 4 Create with no data

```
CREATE MATERIALIZED VIEW project_stats_mv AS
SELECT p.project_name, p.budget, d.dept_name,
       COUNT(e.emp_id) AS assigned_employees
FROM projects p LEFT JOIN departments d USING (dept_id)
LEFT JOIN employees e ON e.dept_id = d.dept_id
GROUP BY p.project_id, p.project_name, p.budget, d.dept_name
WITH NO DATA;
-- Run this expected-error test separately:
SELECT * FROM project_stats_mv;
-- Then populate the materialized view:
REFRESH MATERIALIZED VIEW project_stats_mv;
SELECT * FROM project_stats_mv;
```

Expected error: materialized view "project_stats_mv" has not been populated. REFRESH loads the data. Employee counts represent department staff because no project assignment table is specified.

6 Database roles

6 1 Basic roles

```
CREATE ROLE analyst NOLOGIN;
CREATE ROLE data_viewer LOGIN PASSWORD 'viewer123';
CREATE USER report_user PASSWORD 'report456';
SELECT rolname FROM pg_roles WHERE rolname NOT LIKE 'pg_%';
```

6 2 Specific attributes

```
CREATE ROLE db_creator LOGIN CREATEDB PASSWORD 'creator789';
CREATE ROLE user_manager LOGIN CREATEROLE PASSWORD 'manager101';
CREATE ROLE admin_user LOGIN SUPERUSER PASSWORD 'admin999';
```

6 3 Grant privileges

```
GRANT USAGE ON SCHEMA public TO analyst, data_viewer, report_user;
GRANT SELECT ON employees, departments, projects TO analyst;
GRANT ALL PRIVILEGES ON employee_details TO data_viewer;
GRANT SELECT, INSERT ON employees TO report_user;
```

ALL PRIVILEGES does not make a joined view automatically updatable.

6 4 Team roles and membership

```
CREATE ROLE hr_team NOLOGIN;
CREATE ROLE finance_team NOLOGIN;
CREATE ROLE it_team NOLOGIN;
CREATE USER hr_user1 PASSWORD 'hr001';
CREATE USER hr_user2 PASSWORD 'hr002';
CREATE USER finance_user1 PASSWORD 'fin001';
GRANT hr_team TO hr_user1, hr_user2;
GRANT finance_team TO finance_user1;
GRANT USAGE ON SCHEMA public TO hr_team, finance_team, it_team;
GRANT SELECT, UPDATE ON employees TO hr_team;
GRANT SELECT ON dept_statistics TO finance_team;
```

6 5 Revoke privileges

```
REVOKE UPDATE ON employees FROM hr_team;
REVOKE hr_team FROM hr_user2;
REVOKE ALL PRIVILEGES ON employee_details FROM data_viewer;
```

6 6 Modify role attributes

```
ALTER ROLE analyst LOGIN PASSWORD 'analyst123';
ALTER ROLE user_manager SUPERUSER;
ALTER ROLE analyst PASSWORD NULL;
ALTER ROLE data_viewer CONNECTION LIMIT 5;
```

A NULL password prevents password authentication; LOGIN remains enabled. SUPERUSER changes require a superuser. The supplied passwords and elevated roles are lab examples.

7 Advanced role management

7 1 Role hierarchy

```
CREATE ROLE read_only NOLOGIN;
GRANT USAGE ON SCHEMA public TO read_only;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO read_only;
CREATE ROLE junior_analyst LOGIN PASSWORD 'junior123';
CREATE ROLE senior_analyst LOGIN PASSWORD 'senior123';
GRANT read_only TO junior_analyst, senior_analyst;
GRANT INSERT, UPDATE ON employees TO senior_analyst;
```

7 2 Object ownership

```
CREATE ROLE project_manager LOGIN PASSWORD 'pm123';
GRANT USAGE, CREATE ON SCHEMA public TO project_manager;
GRANT SELECT ON employees, departments TO project_manager;
ALTER VIEW dept_statistics OWNER TO project_manager;
ALTER TABLE projects OWNER TO project_manager;
SELECT tablename, tableowner FROM pg_tables
WHERE schemaname = 'public';
SELECT viewname, viewowner FROM pg_views
WHERE schemaname = 'public' AND viewname = 'dept_statistics';
```

The new view owner needs SELECT on its source tables. pg_tables checks table owners; pg_views checks view owners.

7 3 Reassign and remove a role

```
CREATE ROLE temp_owner LOGIN;
CREATE TABLE temp_table (id INT);
ALTER TABLE temp_table OWNER TO temp_owner;
REASSIGN OWNED BY temp_owner TO postgres;
DROP OWNED BY temp_owner;
DROP ROLE temp_owner;
```

temp_table survives because it now belongs to postgres. DROP OWNED removes remaining privileges and objects still owned by temp_owner in this database.

7 4 Restrict access through views

```
CREATE VIEW hr_employee_view WITH (security_barrier = true) AS
SELECT emp_id, emp_name, dept_id, salary
FROM employees WHERE dept_id = 102;
-- Remove the unrestricted grant from exercise 6.4:
REVOKE SELECT ON employees FROM hr_team;
GRANT SELECT ON hr_employee_view TO hr_team;
CREATE VIEW finance_employee_view AS
SELECT emp_id, emp_name, salary FROM employees;
GRANT SELECT ON finance_employee_view TO finance_team;
```

These views restrict rows or columns. They do not enable PostgreSQL table RLS policies. Restrictions work only when users have no alternative grants exposing the same data.

8 Practical scenarios

8 1 Department dashboard

Assumption: all existing projects count as active because the specified schema has no status column.

```
CREATE VIEW dept_dashboard AS
SELECT d.dept_name, d.location,
       COALESCE(e.n, 0) AS employee_count,
       ROUND(e.avg_salary::numeric, 2) AS average_salary,
       COALESCE(p.n, 0) AS active_projects,
       COALESCE(p.budget, 0) AS total_project_budget,
       ROUND(COALESCE(p.budget, 0)::numeric /
             NULLIF(e.n, 0), 2) AS budget_per_employee
FROM departments d
LEFT JOIN (
  SELECT dept_id, COUNT(*) AS n, AVG(salary) AS avg_salary
  FROM employees GROUP BY dept_id
) e USING (dept_id)
LEFT JOIN (
  SELECT dept_id, COUNT(*) AS n, SUM(budget) AS budget
  FROM projects GROUP BY dept_id
) p USING (dept_id);
```

Budget per employee is NULL when there are no employees; division by zero is avoided.

8 2 High budget project audit

```
ALTER TABLE projects ADD COLUMN created_date
TIMESTAMP DEFAULT CURRENT_TIMESTAMP;
CREATE VIEW high_budget_projects AS
SELECT p.project_name, p.budget, d.dept_name, p.created_date,
       CASE WHEN p.budget > 150000 THEN 'Critical Review Required'
            WHEN p.budget > 100000 THEN 'Management Approval Needed'
            ELSE 'Standard Process' END AS approval_status
FROM projects p LEFT JOIN departments d USING (dept_id)
WHERE p.budget > 75000;
```

8 3 Four access levels

```
CREATE ROLE viewer_role NOLOGIN;
CREATE ROLE entry_role NOLOGIN;
CREATE ROLE analyst_role NOLOGIN;
CREATE ROLE manager_role NOLOGIN;
GRANT USAGE ON SCHEMA public TO viewer_role;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO viewer_role;
GRANT viewer_role TO entry_role;
GRANT INSERT ON employees, projects TO entry_role;
GRANT entry_role TO analyst_role;
GRANT UPDATE ON employees, projects TO analyst_role;
GRANT analyst_role TO manager_role;
GRANT DELETE ON employees, projects TO manager_role;
CREATE USER alice PASSWORD 'alice123';
CREATE USER bob PASSWORD 'bob123';
CREATE USER charlie PASSWORD 'charlie123';
GRANT viewer_role TO alice;
GRANT analyst_role TO bob;
GRANT manager_role TO charlie;
```

Alice can read; Bob inherits read, insert and update; Charlie also gains delete. Grants on ALL TABLES include existing views but do not cover future objects. If INSERT uses SERIAL defaults, grant USAGE on the relevant sequences to the inserting roles.

Conclusion: Regular views run their queries on access; materialized views store results until refresh. Roles organize permissions through grants, revocations and membership.

References: PostgreSQL documentation — CREATE VIEW, REFRESH MATERIALIZED VIEW, GRANT.

https://www.postgresql.org/docs/current/sql-createview.html
https://www.postgresql.org/docs/current/sql-refreshmaterializedview.html
https://www.postgresql.org/docs/current/sql-grant.html